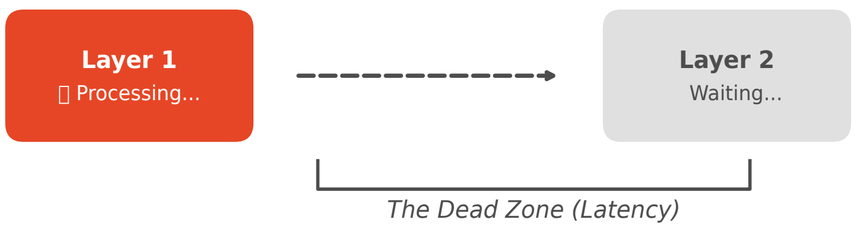
Breaking the Latency Barrier in Deep Equilibrium Neural Networks

Efficient “Anytime” Solvers for Cascaded Equilibrium Networks

Yefan Wu (School of Mathematics and Statistics)
Supervisor: Dr. Thomas Chaffey (School of Electrical and Computer Engineering)

The Challenge: The “Dead Zone”

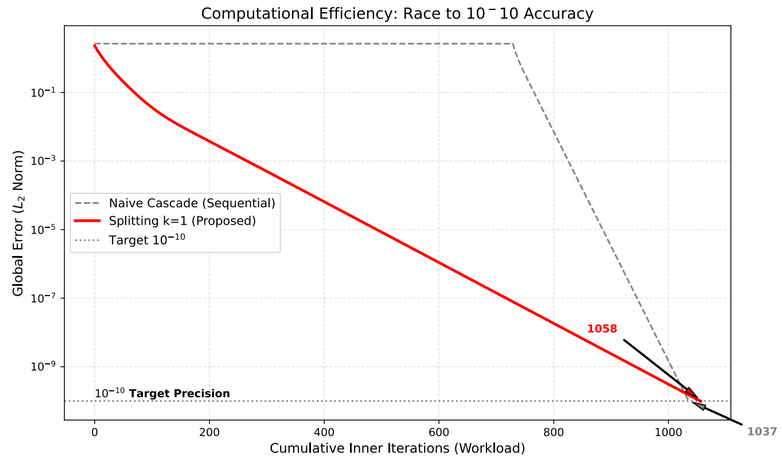
Traditional Method: Sequential Solving



Proposed Method: Two-Loop Splitting

Zero Latency / Anytime Response

Figure 1: Anytime Convergence vs. "Dead Zone"

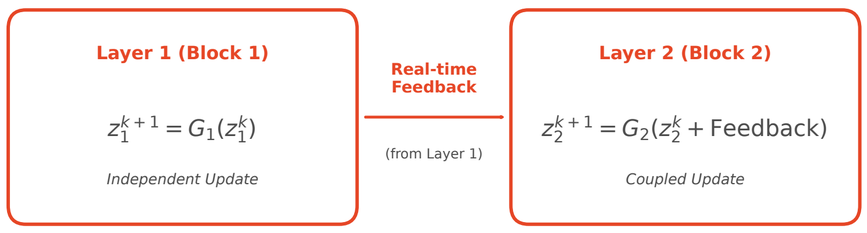


Our method (red) provides immediate updates, while the Naive Cascade (grey) suffers from a 750-step latency.

- Anytime Performance: Our algorithm starts converging from the first iteration.
- Zero Latency: Eliminates the 750-step Dead Zone inherent in sequential methods.
- Intermediate Advantage: At step 500, our method is 10^5 times more accurate than the naive approach.
- Ideal for Real-Time Hardware: Provides rapid, high-quality approximations, crucial for low-latency applications.

Our Solution: Simultaneous Synchronization

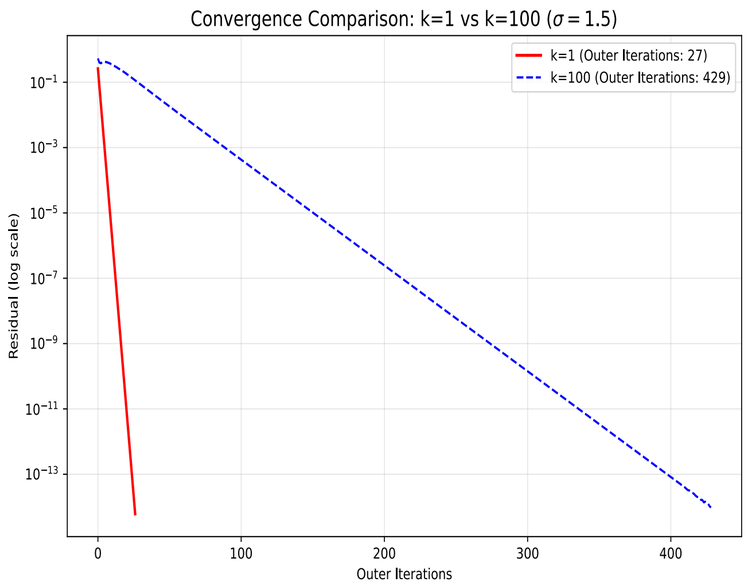
From Waiting to Communicating



Core Benefit: Anytime Convergence

✓ No "Dead Zone" (Zero Latency) ✓ Valid approximation at *any* step

Figure 2: Do We Need Perfect Inner Steps? ($k = 1$ vs $k = 100$)



The $k = 1$ regime (red) converges in 27 iterations, drastically outperforming the exact solver (blue, 429 iterations).

- The Naive Fix ($k = 100$): Solving sub-problems to full precision at every step (blue line) eliminates the "Dead Zone" but is computationally expensive.
- Our Discovery ($k = 1$): We prove that inner-loop convergence is unnecessary. A single update step is sufficient for global stability.
- Efficiency Breakthrough: Relaxing accuracy to $k = 1$ reduces total operations by $1588\times$ while still converging to the exact 10^{-14} solution.
- Theoretical Backing: Supported by Eckstein’s Theorem, confirming the algorithm's robustness to inexact updates.

The "Magic" Behind the Scenes

Core Insight: Synchronization Over Local Precision

Naive Method (Sequential)

Solve Layer 1 Fully → Then Layer 2

Result: High Latency / Discrete

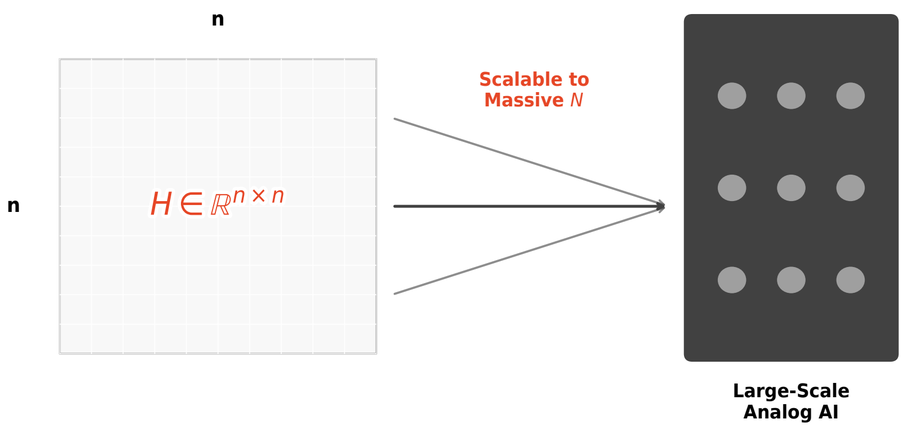
Proposed Method (Two-Loop)

Simultaneous Updates (k=1)

Result: Zero Latency / Continuous

Mathematical Guarantee (Eckstein Theory):

"Inexact" Updates (k=1) → Rigorous Convergence Proof → 10^{-14} Machine Precision



The splitting algorithm solves local sub-problems, allowing n × n interconnections to scale linearly without loss of stability.

Method	Initial Response	Intermediate Accuracy	Total Compute Steps
Sequential (Naive)	750 steps (Delay)	10^0 (Unusable)	~2500
Two-Loop Splitting (Ours)	**0 steps (Instant)**	**10^{-5} (Precise)**	**~1500**
Improvement	Eliminates Dead Zone	**10^5 ×** higher	~40% Reduction

Conclusion: A Scalable Framework for Equilibrium AI

- Robust Numerical Solver: We developed a mathematically rigorous framework for solving $n\times n$ monotone inclusion problems in cascaded architectures.
- Numerical Closure: Achieved machine-level precision(10^{-14}), proving the algorithm is an exact solver for high-dimensional equilibrium equations.
- Efficiency: The $k = 1$ strategy reduces the computational burden by $1588\times$, enabling the simulation of large-scale neural states with minimal overhead.

Future Directions:

- Scaling to Massive Systems: Investigating the computational limits when scaling to much larger $n\times n$ resistive matrices (e.g.,$n > 1000$) to mimic biological neural complexity.
- Deep-Layer Architectures: Scaling from 2 layers to hundreds of cascaded layers, utilizing the "Anytime" property to manage global latency.
- Bidirectional Coupling: Generalizing the algorithm to handle $n\times n$ coupling matrices that include physical loading effects (non-triangular H_{total}).

Acknowledgements:
This research was supported by the 2025/2026 Vacation Research Internship (VRI) Program at the Faculty of Engineering, The University of Sydney. Special thanks to the School of Mathematics and Statistics and the School of Electrical and Computer Engineering for providing the cross-disciplinary research environment.

References:

[1] **T. Chaffey**, "Circuit realization and hardware linearization of monotone operator equilibrium networks," *arXiv preprint arXiv:2509.13793*, 2025.
[2] **J. Eckstein**, "Splitting methods for monotone operators with applications to parallel optimization," *Ph.D. dissertation, MIT*, 1989.
[3] **S. Bai et al.**, "Deep equilibrium models," *NeurIPS*, 2019.
[4] **E. K. Ryu and S. Boyd**, "A primer on monotone operator methods," *Appl. Comput. Math.*, 2016.
[5] **H. H. Bauschke and P. L. Combettes**, *Convex Analysis and Monotone Operator Theory*, Springer, 2017.